KAPU measurements

On a knowledge file the model had not seen (PixelLab documentation, 206,899 tokens), an agent using KAPU read 4,104 tokens per request, an agent searching one Markdown file 8,280, and one searching the same text as a PDF 8,207 (median, the same prompt for all three). Correct answers: KAPU 12, Markdown 12, PDF 11 of 12, by three blind judges.

Latest measurements 2026-10-26 · tools 0.2.0, specification v1.2, file format 1.2 · Windows 11, release build · every number comes from bench/results/ in the repository

KAPU	One .md file + search	PDF + search	Whole file in the context
4,104	8,280	8,207	206,899
tokens per request (median)	tokens per request (median)	tokens per request (median)	tokens on every request

File format 1.2: vectors in the file, replies within a budget

Measured 2026-10-26 with tools 0.2.0, specification v1.2, file format 1.2, on the same corpora, questions and machine as format 1.1, and on a fourth corpus: agent skills and frontend notes with 77 English and 60 Hungarian questions. The search vectors are now in the file, one sign bit per component of every window vector. The sign bits pick the 200 best candidates and the model's float vectors order them, so the first search on another machine needs no cache. Every MCP reply starts with its own price and stays within a max_tokens budget, and kapu_ask searches and opens in one call.

Requirements of the second protocol

The measurable requirements of the 2026-10-26 protocol and their acceptance limits. Tokens: o200k_base.

ID	Requirement	Target	Measured	Met
K-01	Tool descriptions and input schemas, all six tools	≤ 800	797	yes
K-02	First map reply on a file of 3,000 rooms	≤ 1,000	306	yes
K-05	Search reply, 10 hits with excerpts (largest on any corpus)	≤ 600	586	yes
K-07	Questions answered by one kapu_ask call (skills corpus)	≥ 70%	83.1%	yes
K-11	File growth from the #VEC section (largest corpus, MDN)	≤ 3%	3.3%	no
K-12	Hungarian questions on English skills, translated: hybrid recall@10	≥ 0.75	0.78	yes
K-13	The same without translation (needs another embedding model)	≥ 0.60	0.55	no
K-22	Opening a door on a file of 3,000 rooms (median)	< 20 ms	7.6 ms	yes

Four corpora in format 1.2

File size before and after the vector section. Times and reply sizes are medians over MCP, except the largest search reply; tokens are what the agent receives.

Corpus	File, v1.1 (MiB)	File, v1.2 (MiB)	Growth	Search (ms)	Open (ms)	First map (tokens)	Next map (tokens)	Search (tokens)	Largest search (tokens)	kapu_ask (tokens)	Answered by one kapu_ask
Rust book	2.21	2.24	1.7%	32	3.6	328	159	527	580	1,657	21%
Kubernetes	26.22	26.78	2.2%	50	7.9	967	983	528	586	1,795	20%
MDN	128.34	132.52	3.3%	282	48	969	980	500	583	1,664	12%
Skills and notes	5.56	5.67	2.0%	26	2.9	600	224	547	585	1,786	83%

Retrieval: room-level recall@10, hybrid ranking

Sign bits alone lost on MDN, on the skills and most of all with Hungarian questions on English text. With the float rescoring of the best candidates they are at the float level or above everywhere; the price is search time.

Question set	Questions	v1.1, float vectors	v1.2, sign bits alone	v1.2, sign bits + rescoring
Rust book (Stack Overflow)	68	0.485	0.500	0.485
Kubernetes (Stack Overflow)	201	0.458	0.458	0.458
MDN (Stack Overflow)	395	0.271	0.246	**0.278**
Skills, English questions	77	0.948	0.922	**0.961**
Skills, Hungarian questions	60	0.633	0.583	0.633
Skills, the Hungarian questions translated	60	—	0.833	0.833

Not measured yet

- Skill finding with 250 skills (the test machine has 91), and with a prompt that does not mention skills, where the agent has to decide on its own.
- A local model (Ollama) as the agent.
- Clients other than Claude Code: the configurations for the Codex CLI and VS Code are in the README, but they were not tried, and no measurement with them is planned.

Finding skills: native Claude Code skills, one KAPU file, a Markdown folder

12 realistic development tasks (a landing page, an MCP server, a Claude Code plugin, a flaky API, Office files and others), each with 2 to 4 skills fixed in advance as clearly relevant, 39 in all. The same skills in three forms, 45 and 91 of them (91 is every real skill on the test machine). Every task ran once per form and size with opus through claude -p (Claude Code 2.1.310), on 2026-10-26. The prompt points at the skills, as a user would: load every relevant skill with the Skill tool; find and open every relevant skill in the KAPU file; find and read every relevant skill in this folder. A skill counts as found when the Skill tool loaded it, one of its files was read, or one of its rooms was opened.

Results per form

Wrong: loaded although the task neither needs it nor could reasonably use it. Tokens per task: every input token of the run, each turn counted (median). $ per task: what Claude Code reports at API prices, cache reads at their lower price (median). Fixed cost: the first turn of a session that needs no skill; the native skill list is in every session's system prompt.

Skills	Form	Relevant skills found	Tasks with all found	Wrong skills loaded	Tokens per task	$ per task	Calls per task	Seconds per task	Fixed cost per session (tokens)
45	Native skills	38 / 39 (97%)	11 / 12	0	42,574	$0.224	4	9	12,652
45	KAPU file, catalog (0.2.1)	38 / 39 (97%)	11 / 12	3	58,266	$0.131	5.5	25	4,775
45	KAPU file (0.2.0)	37 / 39 (95%)	10 / 12	0	77,215	$0.229	7.5	30	4,774
45	Markdown folder	39 / 39 (100%)	12 / 12	14	93,176	$0.283	10.5	24	5,430
91	Native skills	39 / 39 (100%)	12 / 12	0	44,953	$0.236	4	12	14,081
91	KAPU file, catalog (0.2.1)	38 / 39 (97%)	11 / 12	3	48,972	$0.099	5	22	4,775
91	KAPU file (0.2.0)	38 / 39 (97%)	11 / 12	0	88,418	$0.255	9	33	4,774
91	Markdown folder	39 / 39 (100%)	12 / 12	12	107,726	$0.336	10.5	33	5,430

- With a catalog of the skills in one reply (tools 0.2.1: kapu_map kind=skill, D154), the KAPU file finds as many as before (38 of 39 with 91 skills) in 5 calls instead of 9, with 48,972 input tokens per task (before: 88,418; native skills: 44,953). At $0.099 per task it costs less than the native skills ($0.236), for two reasons: it opens the first part of each skill (up to 2,000 tokens; the rest on request) where the Skill tool loads the whole skill, and less new text is written to the prompt cache, which costs twice the input price, while cache reads cost a tenth.
- The native skills still find the most (39 of 39 with 91 skills) and load nothing wrong. The KAPU file loaded 3 skills that one task did not need, and missed one skill (frontend-expert for the Figma task, which the native skills also missed with 45 skills). The Hungarian-language skill it missed before, it now finds from the catalog.
- The Markdown folder found every skill, but also read 12 skills the tasks did not need (with 91 skills) and cost the most ($0.336 per task).
- The fixed cost of a session: 4,775 tokens with the KAPU tools whatever the number of skills, against 12,652 with 45 and 14,081 with 91 native skills. 250 skills were not measured: the test machine has only 91. Only the KAPU runs were repeated; the native and Markdown runs are those of the first round.

File format 1.1: smaller files, faster calls, better search by meaning

Three changes after the pilot, measured again on the same corpora and questions. The index stores a door's position instead of its id, so files are about 4.5 times smaller and the 13-million-token MDN corpus now builds with the released tool. One call parses and checks the file once and finds index lines by binary search. Search by meaning uses one vector per 512-token window of a door, and the best window counts.

Before and after

Room recall@10: any part of the expected page among the first 10 hits. MDN with v1.0.4 was an experimental build with a 64 MiB line limit.

Metric	Corpus	v1.0.4	v1.1
File size (MiB)	Rust book	7.6	2.2
File size (MiB)	Kubernetes	117.4	26.2
File size (MiB)	MDN	542.2	128.3
Builds with the released tool	MDN	no (E005)	yes
Search over MCP (ms, median)	Rust book	56	12.8
Search over MCP (ms, median)	Kubernetes	657	19.2
Search over MCP (ms, median)	MDN	3,172	158.2
Opening a door over MCP (ms, median)	Rust book	17	2.4
Opening a door over MCP (ms, median)	Kubernetes	216	4.9
Opening a door over MCP (ms, median)	MDN	1,040	44.1
Room recall@10, hybrid search	Rust book	0.5	0.5
Room recall@10, hybrid search	Kubernetes	0.4	0.5
Room recall@10, hybrid search	MDN	0.3	0.3
Room recall@10, vectors only	Rust book	0.3	0.4
Room recall@10, vectors only	Kubernetes	0.3	0.3
Room recall@10, vectors only	MDN	0.2	0.1
kapu bench run time (s)	Kubernetes	187	8
kapu bench run time (s)	MDN	1,774	127

Same content in three formats: KAPU, one Markdown file, a PDF

The PixelLab documentation (third-party, not published), 12 requests of a beginner indie developer. The Markdown and PDF agents searched the text and read only what they found: lines of the Markdown file, pages of the PDF. Pasting the whole file into the conversation would cost 206,899 tokens on every request. Three blind Claude Sonnet judges checked every claim against the documentation.

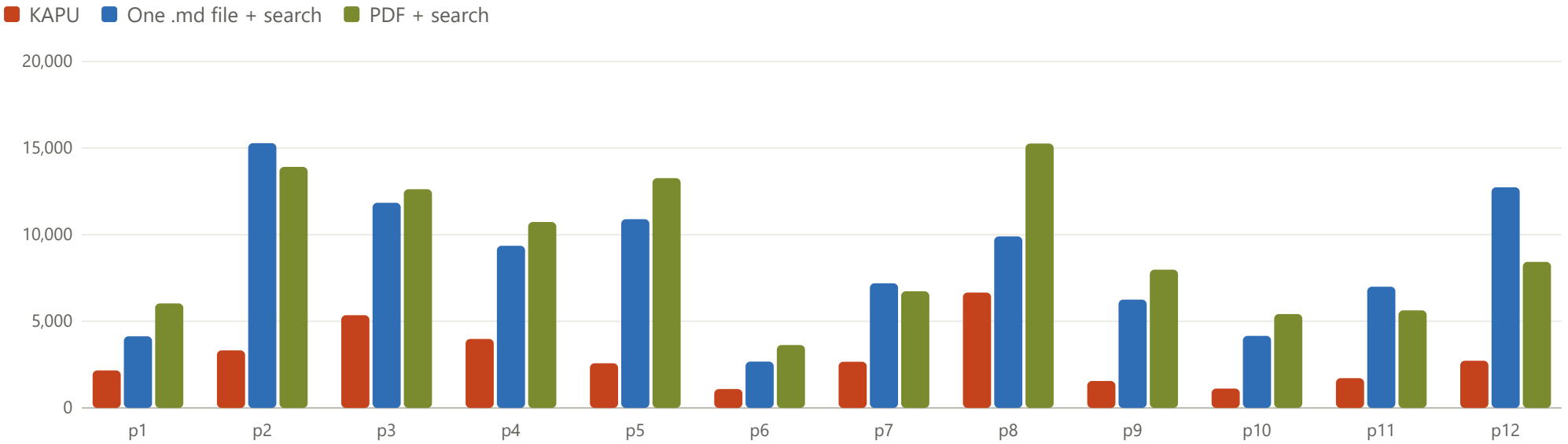
Second round: the same prompt for all three formats

KAPU with specification v1.0.4. The Markdown and PDF answers are those of the first round, scored again by new judges.

Format	Correct	Partly	Supported claims	All claims	Helpfulness (1–5)	Tokens/request	Tokens, 12 requests	Calls/request
KAPU	12	0	153	157	4.17	4,104	48,557	4
One .md file + search	12	0	140	145	4.00	8,280	101,452	8
PDF + search	11	1	147	151	3.83	8,207	109,675	6

First round: tokens read per request

KAPU with specification v1.0.3 and its own prompt. Tokens: what the tools returned to the agent (o200k_base).



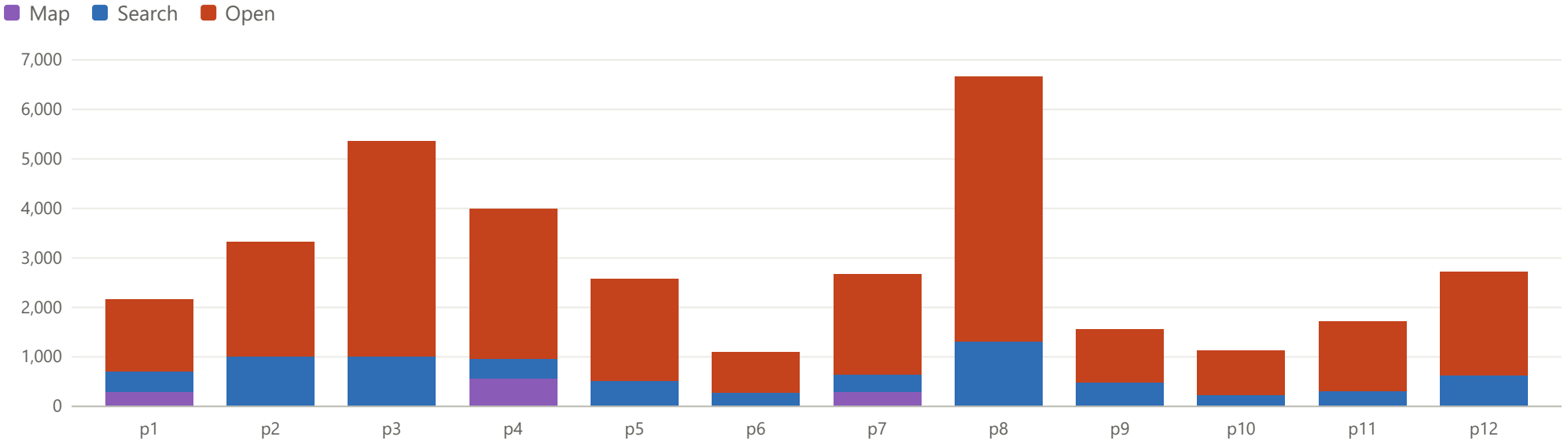
Format	Correct	Partly	Supported claims	All claims	Helpfulness (1–5)	Tokens/request	Tokens, 12 requests	Calls/request
KAPU	12	0	122	123	4.08	2,624	34,953	3
One .md file + search	12	0	134	138	4.25	8,280	101,452	8
PDF + search	11	1	146	151	4.33	8,207	109,675	6

Why thousands of tokens when the map is a few hundred?

The map and the search are the table of contents and the hit list. Most tokens come from the opened doors: the documentation text the answer is written from. Of the KAPU tokens of the 12 requests, opening doors took 77%, searching 20%, the map 3%.

KAPU tokens per request, by call

A typical request: one or two searches (300–400 tokens), one or two opened doors (1,000–3,000 tokens), rarely the map.



Scale: small, medium and large documentation

S: the Rust book, M: the Kubernetes documentation, L: MDN Web Docs, all in format 1.1. Over one MCP session: the map, every group list, 20 searches with room titles as queries and an open of each top hit.

Build, size and responses

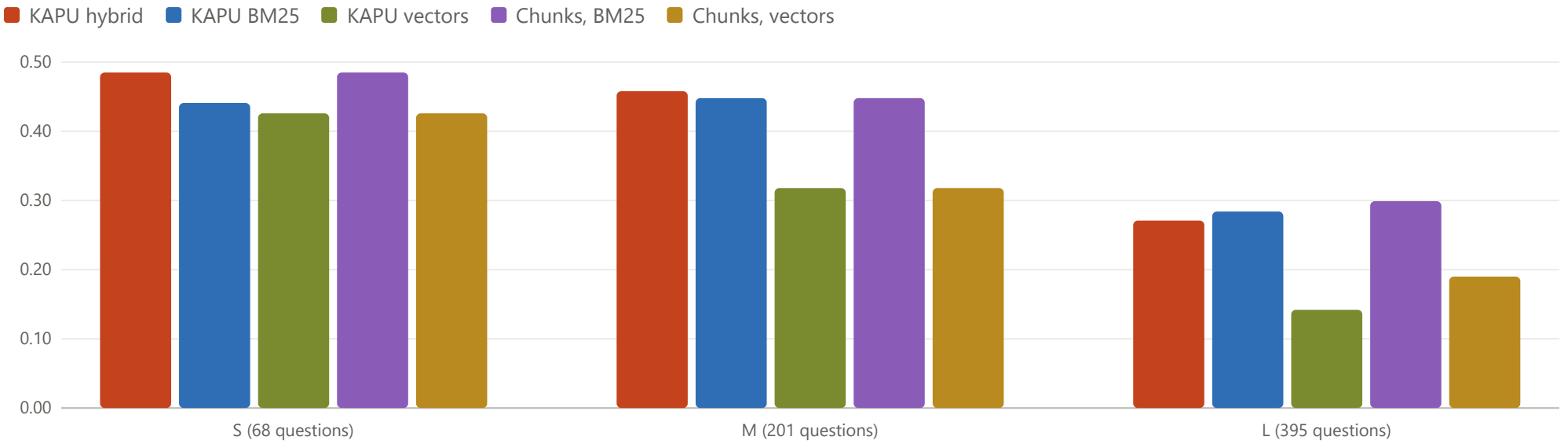
Size	Corpus	Knowledge (tokens)	Rooms	Doors	Groups	Build (s)	Memory (GB)	File (MiB)	Map (tokens)	First map (tokens)	Largest group list (tokens)	Search (tokens)	Search (ms)	First search (s)	Open (tokens)	Open (ms)
S	Rust book	321,618	111	231	7	4.6	0.69	2.2	153	322	804	246	12.8	1.1	1,689	2.4
M	Kubernetes	3,873,333	1,722	6,370	104	38	1.03	26.2	1,950	2,318	6,957	260	19.2	1	240	4.9
L	MDN	12,995,136	14,719	84,910	99	240.3	2.68	128.3	1,921	2,303	30,118	212	158.2	1.3	366	44.1

Retrieval without an agent: KAPU against chunked RAG

Real Stack Overflow questions whose accepted answer links exactly one page of the documentation: 68, 201 and 395 questions. A hit is any part of that page among the first results. The chunk baselines use 512-token chunks with 64 tokens of overlap and the same embedding model.

Room recall@10 by size and method

KAPU returns doors (rooms and their sections), the baselines return chunks.



Size	Method	Questions	Recall@5	Recall@10	MRR@10
S	KAPU hybrid	68	0.397	0.485	0.281
S	KAPU BM25	68	0.338	0.441	0.259
S	KAPU vectors	68	0.353	0.426	0.184
S	Chunks, BM25	68	0.426	0.485	0.282
S	Chunks, vectors	68	0.368	0.426	0.230
M	KAPU hybrid	201	0.373	0.458	0.245
M	KAPU BM25	201	0.348	0.448	0.218
M	KAPU vectors	201	0.219	0.318	0.147
M	Chunks, BM25	201	0.338	0.448	0.218
M	Chunks, vectors	201	0.249	0.318	0.145
L	KAPU hybrid	395	0.172	0.271	0.128
L	KAPU BM25	395	0.220	0.284	0.135
L	KAPU vectors	395	0.086	0.142	0.055
L	Chunks, BM25	395	0.230	0.299	0.159

Size	Method	Questions	Recall@5	Recall@10	MRR@10
L	Chunks, vectors	395	0.124	0.190	0.081

Live run on the Rust book: five systems, 30 questions

The model answered these popular questions well from memory too (closed book: 30 of 30), so here accuracy does not separate the systems; the cost does. KAPU with specification v1.0.3; with v1.0.4 the KAPU agents read a median of 2,331 tokens per question at the same accuracy.

Correct answers and cost per system

Two blind judges (Claude Sonnet and Haiku). All agent tokens: the whole run divided by its questions, an estimate.

System	Correct (Sonnet judge)	Correct (Haiku judge)	Questions	Tokens read/question	Calls/question	All agent tokens/question
Closed book, no tools	30	28	30	0	0	2,706
Agentic chunked RAG	30	27	30	4,150	1	12,648
grep/read on the Markdown folder	29	28	30	1,290	2	9,343
llms.txt	30	29	30	8,738	2	11,615
KAPU	30	29	30	3,982	2	10,118

Earlier fix round: specification v1.0.4

Metric	v1.0.3	v1.0.4
KAPU on the Rust book: tokens read per question (median)	3,982	2,331
KAPU on the Rust book: correct answers (of 30)	30	30
First map call, Kubernetes (tokens)	4,989	2,318
First map call, MDN (tokens)	4,139	2,303

Method and limits

- Corpora: the Rust book (MIT or Apache-2.0), the Kubernetes documentation (CC BY 4.0) and MDN Web Docs (CC BY-SA 2.5) at pinned commits. Questions: Stack Overflow API (CC BY-SA); only their ids are stored.
- Agents: Claude Sonnet subagents that could only use logged tools. Tokens are counted with o200k_base.
- Judges: blind Claude models; the answers were shuffled under random letters, and source lines were removed.
- This is a pilot on one machine (Windows 11, release build), not a benchmark. The PixelLab file is third-party and is not published.

Repository RESULTS.md Raw numbers (JSON)